oauth2-passkey

Passwordless Authentication Library for Rust

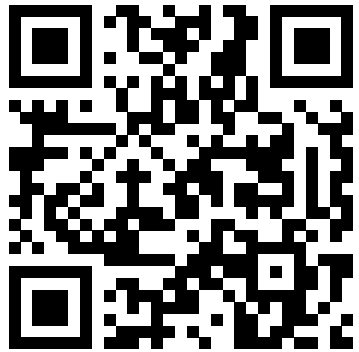
Kimitoshi Takahashi

Tokyo Rust Show & Tell | 2026/03/31

Live Demo

Demo

1. **Google OAuth2** - First-time login with Google
2. **Passkey Promotion** - Library prompts to register fingerprint/face
3. **Passkey-only Login** - Next time, just biometrics. No redirect.
4. **Account Linking** - Both methods, same user



passkey-demo.ccmp.jp

What the Demo Showed

Feature	Details
OAuth2/OIDC	Google login (also works with FedCM)
Passkey Promotion	After OAuth2 login, prompts user to register Passkey
Passkey	Google Password Manager, Apple, Windows Hello, bitwarden, Proton Pass, YubiKey
Account Linking	OAuth2 + Passkey mapped to same user
Built-in UI	Login page, account management, admin panel included
Session	Cookie-based session with CSRF protection

All handled by a single library: `oauth2-passkey`

Motivation

I wanted to build **exactly what you just saw** — in Rust.

- **OAuth2/OIDC**: delegate auth to trusted providers — no passwords to manage
- **Passkey**: phishing-resistant, biometrics/hardware key — no server-side secrets
- No integrated Rust/Axum library existed → built one → published to **crates.io**

Agenda

1. **How OAuth2 & Passkey Work**
2. **Using the Library**
3. **Internals of Multi Storage Support**
4. **Integrating with Your App**
5. **Wrap-up**

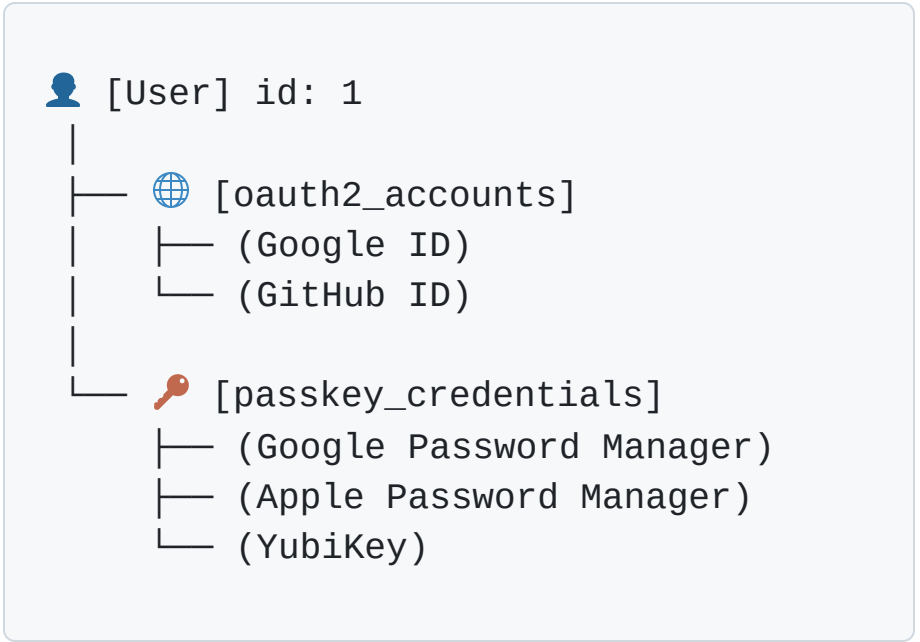
Flow: Google Login → Passkey Setup

Step-by-step

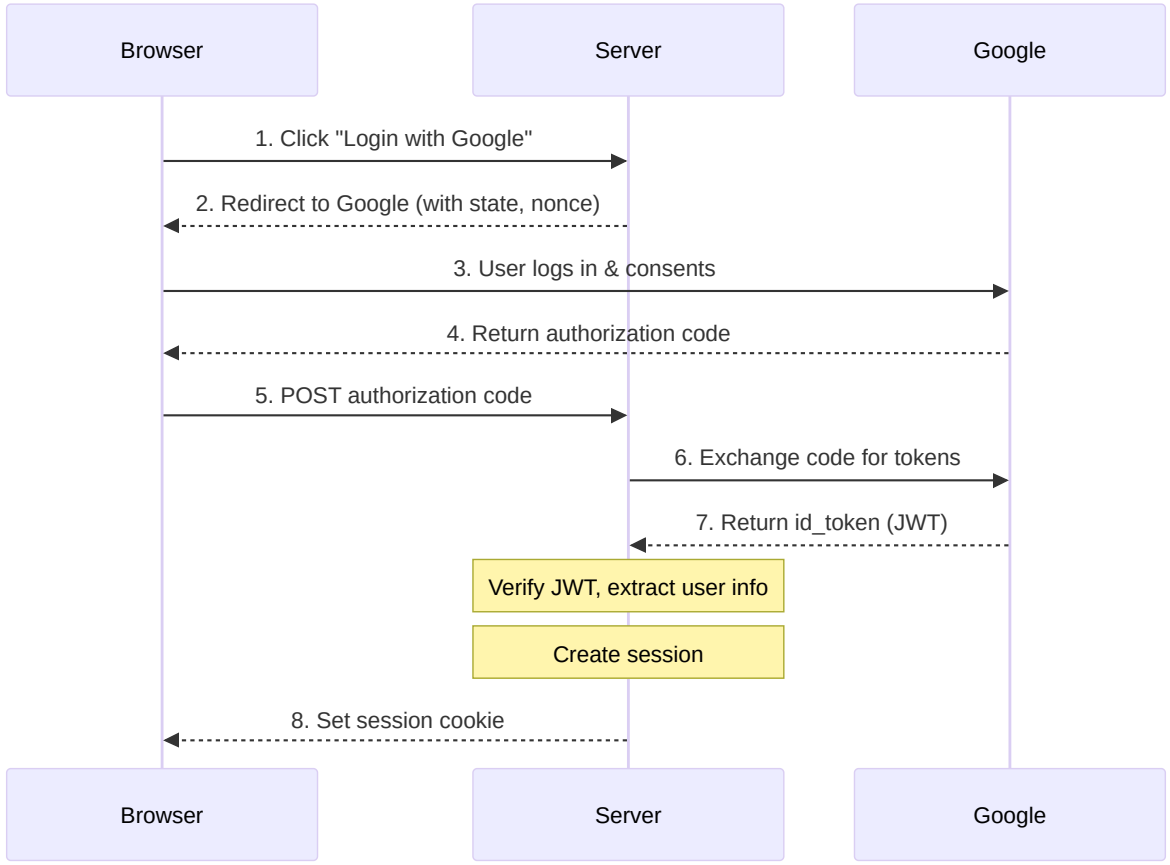
1. **OAuth2 Login**
   ➔ Create a user in db linked with Google account
2. **Passkey Promotion**
   ➔ Register Passkey ➔ link to the user
3. **Next Login**
   ➔ Login with either Passkey or Google OAuth2

One user can have multiple OAuth2 accounts and multiple passkey credentials.

Internal Result (Linking)

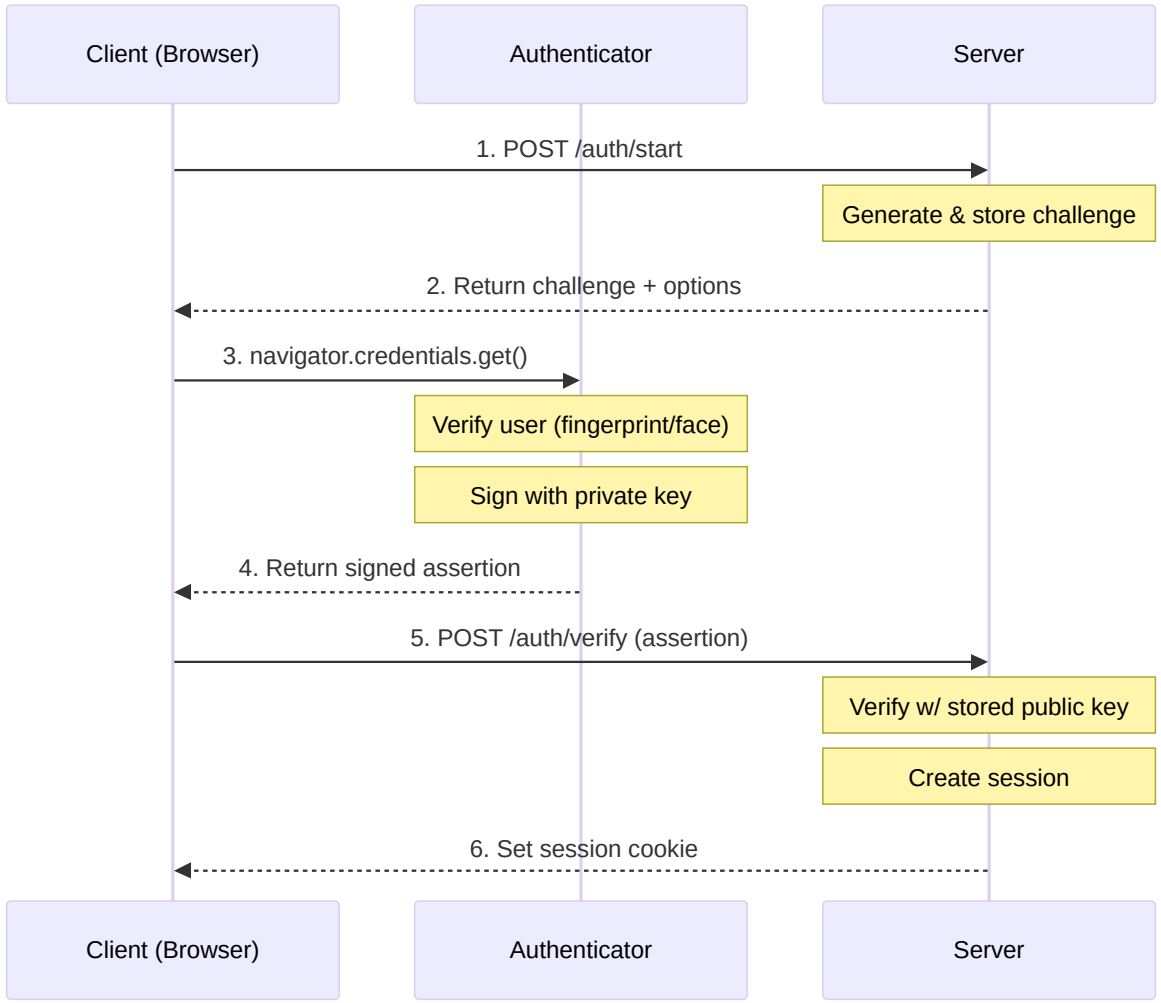


How OAuth2/OIDC Works



Page-redirect: Google → code → id_token → session

How Passkey/WebAuthn Works



JavaScript-driven: challenge → sign → verify → session

Authenticators: Google Password Manager, Apple, Windows Hello, YubiKey, …

Using the Library

.env Setup (Minimal)

```
ORIGIN='http://localhost:3001'

# Google OAuth2 credentials
OAUTH2_GOOGLE_CLIENT_ID='xxx.apps.googleusercontent.com'
OAUTH2_GOOGLE_CLIENT_SECRET='xxx'

# SQLite + in-memory cache (no DB setup required)
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'
GENERIC_CACHE_STORE_TYPE=memory
GENERIC_CACHE_STORE_URL='memory'
```

- Data store: SQLite, PostgreSQL, MySQL
- Cache store: memory, Redis

How to Use

```rust
use oauth2_passkey_axum::{
    AuthUser, oauth2_passkey_full_router, // 1, Import
};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    dotenv().ok();
    oauth2_passkey_axum::init().await?;       // 2. Initialize

    let app = Router::new()
        .route("/", get(index))
        .merge(oauth2_passkey_full_router()); // 3. Merge router

    // Auth endpoints are now at /o2p/*
    spawn_http_server(3001, app).await?;
    Ok(())
}
```

Under `/o2p/*`, we have:

- OAuth2 endpoints and Passkey endpoints
- Built-in login UI, account management, and admin panel

Page Protection: AuthUser Extractor

```rust
// Required auth - redirects to login if not authenticated
async fn protected(user: AuthUser) -> impl IntoResponse {
    format!("Hello, {}!", user.label)
}

// Optional auth - allows anonymous access
async fn public(user: Option<AuthUser>) -> impl IntoResponse {
    match user {
        Some(u) => format!("Hello, {}!", u.label),
        None    => "Hello, anonymous!".to_string(),
    }
}
```

Implemented via Axum's `FromRequestParts` trait:

- `AuthUser` → GET: redirect to login, others: 401
- `Option<AuthUser>` → `OptionalFromRequestParts` impl returns `None` on failure — no redirect

Limitation: always hits DB, GET always redirects. Middleware gives more control: skip DB, or return 401 even on GET.

Page Protection: Middleware

Middleware gives more control: **skip DB, or return 401 even on GET.**

```
let app = Router::new()
    .route("/api/1", get(h1)
        .route_layer(from_fn(is_authenticated_401)))        // <-- No DB query
    .route("/api/2", get(h2)
        .route_layer(from_fn(is_authenticated_user_401)));  // <-- DB query, user info available
```

```
// In handler: access user or CSRF token via Extension
async fn h1(Extension(csrf): Extension<CsrfToken>) { ... }
async fn h2(Extension(user): Extension<AuthUser>) { ... }
```

Page Protection: Middleware Variants

Variant	Unauthenticated	DB Query	Handler Extension
`is_authenticated_401`	401	No	`CsrfToken`
`is_authenticated_user_401`	401	Yes	`AuthUser`
`is_authenticated_redirect`	Redirect to login	No	`CsrfToken`
`is_authenticated_user_redirect`	Redirect to login	Yes	`AuthUser`

Storage & LazyLock Pattern

Switch DB by Changing .env

```
# SQLite (dev/demo - no setup required)
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'

# PostgreSQL
GENERIC_DATA_STORE_TYPE=postgres
GENERIC_DATA_STORE_URL='postgres://user:pass@localhost:5432/mydb'

# MySQL / MariaDB
GENERIC_DATA_STORE_TYPE=mysql
GENERIC_DATA_STORE_URL='mysql://user:pass@localhost:3306/mydb'

# Cache: in-memory or Redis
GENERIC_CACHE_STORE_TYPE=memory       # or: redis
GENERIC_CACHE_STORE_URL='memory'      # or: redis://localhost:6379
```

No code changes. Just swap the env vars and restart.

How Multi-DB Support Works Internally

1. LazyLock reads env at startup:

```rust
static GENERIC_DATA_STORE: LazyLock<Mutex<Box<dyn DataStore>>>
    = LazyLock::new(|| {
        let db_type = env::var("GENERIC_DATA_STORE_TYPE");
        let db_url = env::var("GENERIC_DATA_STORE_URL");

        match db_type {
            "sqlite"   => Box::new(SqliteDataStore { pool }),
            "postgres" => Box::new(PostgresDataStore { pool }),
            "mysql"    => Box::new(MySqlDataStore { pool }),
        }
    });
```

2. Trait + impl per backend:

```rust
// DataStore: Send + Sync — safe to hold in a global LazyLock

impl DataStore for SqliteDataStore {
    fn as_sqlite(&self) -> Option<&Pool<Sqlite>> { Some(&self.pool) }
    fn as_postgres(&self) -> Option<&Pool<Postgres>> { None }
    fn as_mysql(&self) -> Option<&Pool<MySql>> { None }
}
```

3. Callers dispatch via trait:

```rust
let store = GENERIC_DATA_STORE.lock().await;
match (store.as_sqlite(), store.as_postgres(), store.as_mysql()) {
    (Some(pool), _, _) => get_all_users_sqlite(pool).await,
    (_, Some(pool), _) => get_all_users_postgres(pool).await,
    (_, _, Some(pool)) => get_all_users_mysql(pool).await,
    _ => Err(UserError::Storage("Unsupported db".into())),
}
```

- `GENERIC_DATA_STORE_TYPE=sqlite` → `LazyLock` holds `SqliteDataStore`
- `SqliteDataStore` returns `(Some, None, None)` for `(as_sqlite, as_postgres, as_mysql)`
- match selects `(Some(pool), _, _)` → `get_all_users_sqlite(pool)` runs

Why LazyLock Instead of Axum State?

If the library used State:

```
// User must compose AuthState into AppState
struct AppState {
    auth: AuthState,  // library's state
    pool: PgPool,     // user's own state
}
let app = Router::new().with_state(state);
```

oauth2-passkey: LazyLock globals

```
// User just calls init()
oauth2_passkey_axum::init().await?;

// Library internally:
let store = GENERIC_DATA_STORE
    .lock().await;
// No state parameter needed
```

For users: No need to worry about library state

For library development: No state threading — any function can access DB directly

Integrating with Your App

Integrating Your App: 1:N Schema (demo-todo)

```sql
CREATE TABLE todos (
    id        SERIAL PRIMARY KEY,
    user_id   TEXT NOT NULL,        -- FK to users.id
    title     TEXT NOT NULL,
    completed BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_todos_user_id ON todos(user_id);
```

- `user_id` links to `users.id` managed by oauth2-passkey — get it from `AuthUser.id`
- Index on `user_id` is essential for efficient per-user queries
- Library DB and app DB can be the same or separate

Integrating Your App: 1:N Handler (demo-todo)

```rust
// main.rs
oauth2_passkey_axum::init().await?;
let pool = db::init_db().await?;
let app = Router::new()
    .merge(todos_router())              // ← protected routes
    .with_state(AppState { pool })
    .merge(oauth2_passkey_full_router());
```

```rust
// handlers.rs
pub fn todos_router() -> Router<AppState> {
    Router::new()
        .route("/todos", get(list_todos).post(create_todo))
        .route_layer(from_fn(is_authenticated_redirect))
}
```

```rust
async fn create_todo(
    State(state): State<AppState>,
    user: AuthUser,
    Form(form): Form<TodoForm>,
) -> Result<Response, ...> {
    db::create_todo(
        &state.pool,
        &user.id,     // ──> saved as todos.user_id
        &form.title,
    ).await?;
    Ok(Redirect::to("/").into_response())
}
```

- Your `AppState` holds your own DB pool — independent from oauth2-passkey's storage
- Protect routes with middleware injecting `AuthUser` — no extra DB query at route level
- Pass `user.id` as FK when writing to your DB

Integrating Your App: 1:1 Schema (demo-profile)

```sql
CREATE TABLE user_profiles (
    user_id      TEXT PRIMARY KEY,  -- PK = FK: enforces 1:1
    display_name TEXT,
    bio          TEXT,
    avatar_url   TEXT,
    theme        TEXT DEFAULT 'light',
    created_at   TIMESTAMPTZ DEFAULT NOW(),
    updated_at   TIMESTAMPTZ DEFAULT NOW()
);
```

- `user_id TEXT PRIMARY KEY` — both PK and FK, enforces 1:1 at DB level
- No separate `id` column needed
- Profile auto-created on first login

Integrating Your App: 1:1 Handler (demo-profile)

```rust
// main.rs
oauth2_passkey_axum::init().await?;
let pool = db::init_db().await?;
let app = Router::new()
    .merge(profile_router())            // ← protected routes
    .with_state(AppState { pool })
    .merge(oauth2_passkey_full_router());
```

```rust
// handlers.rs
pub fn profile_router() -> Router<AppState> {
    Router::new()
        .route("/profile", get(show_profile).post(update_profile))
        .route_layer(from_fn(is_authenticated_redirect))
}
```

```rust
async fn update_profile(
    State(state): State<AppState>,
    user: AuthUser,
    Form(form): Form<ProfileForm>,
) -> Result<Response, ...> {
    db::upsert_profile(
        &state.pool,
        &UserProfile {
            user_id: user.id.clone(), // ——> user_profiles.user_id
            bio: form.bio,
            ..Default::default()
        },
    ).await?;
    Ok(Redirect::to("/").into_response())
}
```

- Your `AppState` holds your own DB pool — independent from oauth2-passkey's storage
- Protect routes with middleware injecting `AuthUser` — no extra DB query at route level
- Pass `user.id` as FK when writing to your DB

Wrap-up

Summary

- **Easy**: Add passwordless auth to your Axum app in minutes — Built-in UI included
- **Secure**: Passkey (phishing-resistant) + OAuth2, with CSRF protection and secure session cookies
- **Flexible**: Switch between SQLite, PostgreSQL, MySQL, and Redis with a single `.env` change

Thank You! / Questions?

About me:

- **Kimitoshi Takahashi**
- Self-employed, reskilling in Rust (3rd year)
- Let's start a startup together!

GitHub

Contact

Extra Slides

Why Not Axum State: The State Threading Problem

With Axum State — state must flow through every layer:

```rust
// parent doesn't use DB, but must accept State
// just to pass it down to grandchild
async fn parent(
    State(state): State<AppState>,
) -> impl IntoResponse {
    child(state).await
}

async fn child(state: AppState) -> impl IntoResponse {
    grandchild(&state).await
}

// Only grandchild actually needs DB
async fn grandchild(state: &AppState) -> impl IntoResponse {
    let users = db::get_users(&state.auth_pool).await;
    // ...
}
```

With LazyLock — no threading needed:

```rust
// parent and child have no idea DB exists
async fn parent() -> impl IntoResponse {
    child().await
}

async fn child() -> impl IntoResponse {
    grandchild().await
}

// grandchild accesses DB directly
async fn grandchild() -> impl IntoResponse {
    let store = GENERIC_DATA_STORE.lock().await;
    // ...
}
```

The library has 80+ internal functions. With State, all of them would need `&State` — even those that just call something else.

AuthUser: FromRequestParts Implementation

```rust
impl<B> FromRequestParts<B> for AuthUser
where
    B: Send + Sync,
{
    type Rejection = AuthRedirect;

    async fn from_request_parts(
        parts: &mut Parts, _: &B,
    ) -> Result<Self, Self::Rejection> {
        let method = parts.method.clone();

        // 1. Extract session cookie
        let session_cookie = cookies
            .get(SESSION_COOKIE_NAME.as_str())
            .ok_or_else(|| AuthRedirect::new(method.clone()))?;

        // 2. Validate session → get user
        let (session_user, csrf_token) =
            get_user_and_csrf_token_from_session(...)
                .await
                .map_err(|_| AuthRedirect::new(method.clone()))?;

        Ok(AuthUser::from(session_user))
    }
}
```

```rust
// AuthRedirect: redirect on GET, 401 otherwise
impl IntoResponse for AuthRedirect {
    fn into_response(self) -> Response {
        if self.method == Method::GET {
            Redirect::temporary(LOGIN_URL).into_response()
        } else {
            (StatusCode::UNAUTHORIZED,
             "Unauthorized").into_response()
        }
    }
}
```

```rust
// Option<AuthUser>: explicit OptionalFromRequestParts
impl<B> OptionalFromRequestParts<B> for AuthUser
where
    B: Send + Sync,
{
    type Rejection = AuthRedirect;

    async fn from_request_parts(...)
        -> Result<Option<Self>, Self::Rejection>
    {
        let result =
            <AuthUser as FromRequestParts<B>>
                ::from_request_parts(parts, state).await;
        Ok(result.ok()) // Err → None, no redirect
    }
}
```

What is LazyLock?

`std::sync::LazyLock` — a value initialized **once**, on first access, then shared immutably. Thread-safe. Stable since Rust 1.80.

```
use std::sync::LazyLock;

// Evaluated once, on first access. Never again.
static CONFIG: LazyLock<String> = LazyLock::new(|| {
    std::env::var("MY_CONFIG").expect("MY_CONFIG must be set")
});

fn anywhere() {
    println!("{}", *CONFIG);  // Just use it. No parameter needed.
}
```

Like `lazy_static!` but in std. No macro, no extra crate.

LazyLock: Benefits & Trade-offs

Benefits

- **Zero boilerplate for users** - no `AppState` struct to create
- **No state threading** - 80+ internal functions access storage directly
- **Env-only config** - switch DB by changing `.env`, no code changes
- **Fail-fast init** - `init().await?` panics on invalid env at startup

Trade-offs

- Single instance per process (fine for auth library)
- Implicit dependencies (function signatures don't show DB access)
- Test isolation needs `#[serial]` (shared global state)

An intentional design trade-off: Rust purists would use State, but LazyLock keeps complexity inside the library.

Middleware: Why Not Just Use the Extractor?

AuthUser extractor:

- Always redirects on auth failure
- Always fetches user from DB
- Great for web pages that need user info

Problem: APIs should return 401 JSON, not redirect HTML. And sometimes you just need to check "is this user logged in?" without a DB query.

Middleware gives 2 axes of control:

Response type:

- `_redirect` → Web pages (GET redirects to login)
- `_401` → APIs (always returns 401)

Extension injected:

- `_user` → DB query, `Extension<AuthUser>`
- Non-`_user` → No DB query, `Extension<CsrfToken>` only

```
// Handler with _user middleware
async fn page(
    Extension(user): Extension<AuthUser>,
) -> impl IntoResponse { ... }

// Handler with non-_user middleware
async fn api(
    Extension(csrf): Extension<CsrfToken>,
) -> impl IntoResponse { ... }
```

Newtype Wrappers: Type Safety

```rust
pub struct CsrfToken(String);
pub struct UserId(String);
pub struct SessionId(String);
pub struct SessionCookie(String);
pub struct CsrfHeaderVerified(pub bool);
pub struct AuthenticationStatus(pub bool);
```

```rust
// Without newtypes - compiles but wrong!
fn check(session_id: &str, user_id: &str);
check(user_id, session_id); // Oops!

// With newtypes - compile error!
fn check(session_id: SessionId,
         user_id: UserId);
check(user_id, session_id); // Error!
```

Prevent mixing up strings/bools at compile time.

Newtype Wrappers: Validation in Constructors

```
impl UserId {
    pub fn new(id: String) -> Result<Self, SessionError> {
        if id.is_empty() {
            return Err(SessionError::Validation("User ID cannot be empty".into()));
        }
        if id.len() > 255 {
            return Err(SessionError::Validation("User ID too long".into()));
        }
        if !id.chars().all(|c| c.is_ascii_alphanumeric()
            || matches!(c, '-' | '_' | '.' | '@' | '+')) {
            return Err(SessionError::Validation("Invalid characters".into()));
        }
        Ok(UserId(id))
    }
}
```

If you have a `UserId`, it's guaranteed valid. No need to re-validate.

Error Hierarchy with thiserror

```rust
#[derive(Error, Debug)]
pub enum CoordinationError {
    #[error("Unauthorized access")]
    Unauthorized,
    #[error("Resource not found: {resource_type} {resource_id}")]
    ResourceNotFound { resource_type: String, resource_id: String },

    // Wrap module-specific errors
    #[error("OAuth2 error: {0}")]   OAuth2Error(OAuth2Error),
    #[error("Passkey error: {0}")]  PasskeyError(PasskeyError),
    #[error("Session error: {0}")]  SessionError(SessionError),
    #[error("User error: {0}")]     UserError(UserError),
}
```

Each module has its own error type. The coordination layer wraps them all.

From Impls with Auto-Logging

```rust
impl From<OAuth2Error>
    for CoordinationError
{
    fn from(err: OAuth2Error) -> Self {
        let error = Self::OAuth2Error(err);
        tracing::error!("{}", error);
        error  // Auto-log on conversion!
    }
}
```

Every `?` that crosses a module boundary automatically logs:

```rust
// ? triggers From impl + logging
let token = exchange_code(code)
    .await?; // OAuth2Error -> logged
let user = get_user(id)
    .await?; // UserError -> logged
```

No manual `tracing::error!()` needed.

Security: Constant-Time Comparison

```
use subtle::ConstantTimeEq;

// CSRF token verification
if header_csrf_token
    .as_bytes()
    .ct_eq(session_csrf_token
        .as_str().as_bytes())
    .into()
{
    // Token matches
}
```

Why not just `==`?

- Regular `==` short-circuits on first mismatch
- Attacker measures response time to guess bytes
- `ct_eq` always takes the same time

Used for CSRF tokens, session validation, and other security-sensitive comparisons.

Atomic SQL: No Explicit Transactions

```rust
// "Delete user, but only if they're not the last admin"
// Done in a SINGLE SQL statement - no transaction needed

pub async fn delete_user_if_not_last_admin_postgres(
    pool: &Pool<Postgres>, id: UserId,
) -> Result<bool, UserError> {
    let result = sqlx::query(&format!(
        r#"DELETE FROM {table}
           WHERE id = $1
           AND (SELECT COUNT(*) FROM {table}
                WHERE is_admin = true) > 1"#
    ))
    .bind(id.as_str())
    .execute(pool).await?;

    Ok(result.rows_affected() > 0)
}
```

Business logic **in SQL** = atomic by default. No race conditions.

SQL Dialect Differences

PostgreSQL - one query

```
sqlx::query_as::<_, User>(&format!(
    "INSERT INTO {table} (...)
     VALUES ($1, $2, ...)
     ON CONFLICT (id) DO UPDATE SET ...
     RETURNING *"
)).fetch_one(pool).await
```

SQLite - need two queries

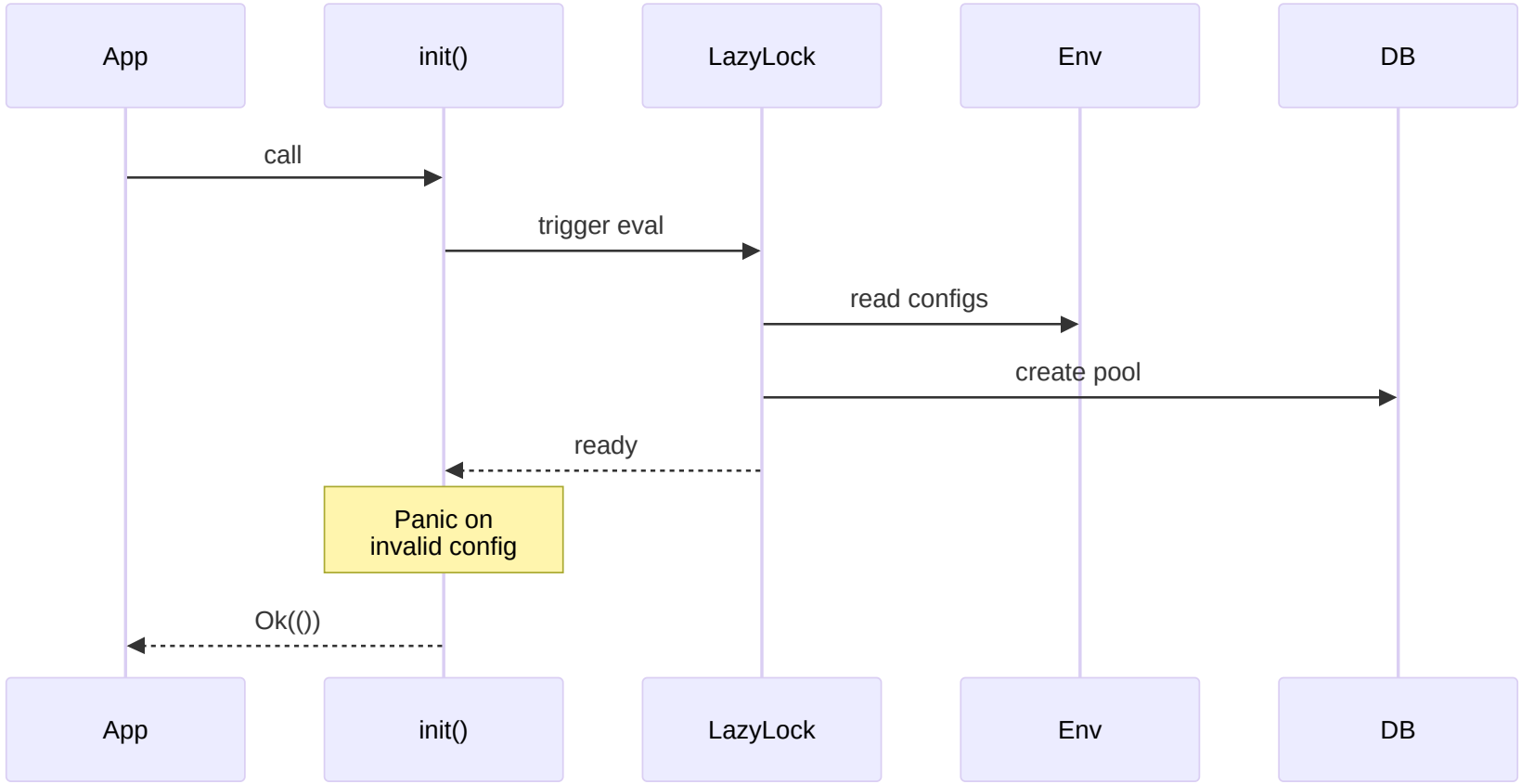
```
sqlx::query(&format!(
    "INSERT INTO {table} (...)
     VALUES (?, ?, ...)
     ON CONFLICT (id) DO UPDATE SET ..."
)).execute(pool).await?;

sqlx::query_as::<_, User>(&format!(
    "SELECT * FROM {table} WHERE id = ?"
)).fetch_one(pool).await
```

Supporting 3 databases means handling these quirks per-backend.

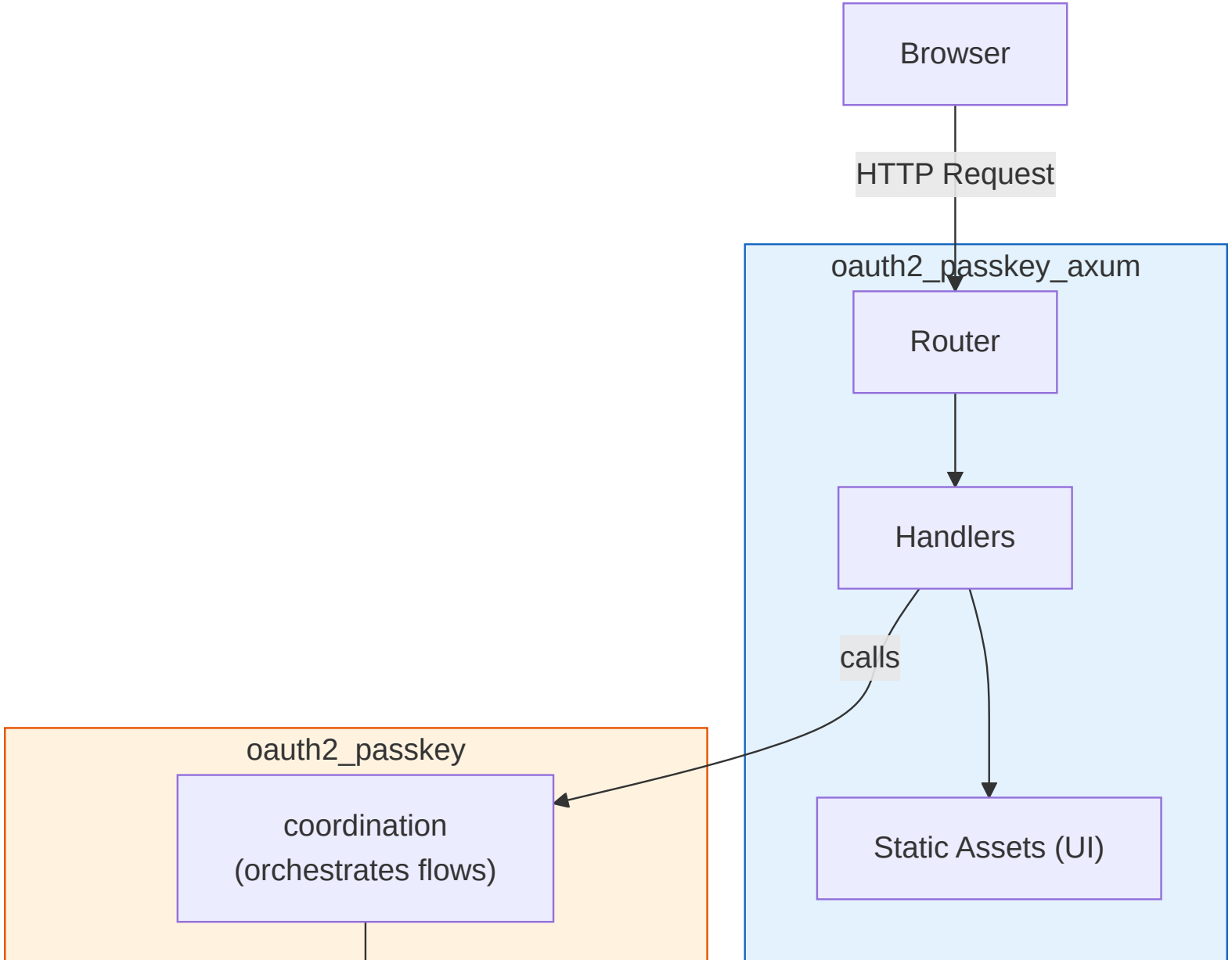
LazyLock Initialization Flow

Fail-fast at Startup: Evaluates all configs/DB immediately.

No Axum State: Any handler can safely access `GENERIC_DATA_STORE` globally.



Crate Structure (Detail)



From/Into: Clean Layer Boundaries

```rust
// DB layer -> Session layer
impl From<DbUser> for SessionUser {
    fn from(db_user: DbUser) -> Self {
        Self {
            id: db_user.id,
            account: db_user.account,
            label: db_user.label,
            is_admin: db_user.is_admin,
            ..
        }
    }
}
```

```rust
// Session -> Axum layer
impl From<SessionUser> for AuthUser {
    fn from(su: SessionUser) -> Self {
        AuthUser {
            id: su.id,
            // ... copy fields
            csrf_token: String::new(),
            session_id: String::new(),
            // ^ Axum-specific fields
        }
    }
}
```

Each layer has its own type. `From` impls make conversion seamless with `into()`.

Future Plans

- **DPoP (Demonstration of Proof-of-Possession)**
  - Bind tokens to a cryptographic key
  - Prevent token theft/replay
- **Bearer Token support**
  - API authentication beyond session cookies
- **FedCM (Federated Credential Management)**
  - Browser-native identity UI (experimental, already partially implemented)
- **More OAuth2 providers**
  - GitHub, Apple, Microsoft, etc.